

Examen Réparti 2 PSCR Master 1 Informatique Jan 2024

UE MU4IN400

Année 2023-2024

2 heures – **Tout document papier autorisé sauf corrigés fournis de l'UE imprimés (annales, TD, TME)**
Tout appareil de communication électronique interdit (téléphones...)

Introduction

- Le barème est sur 23 points et est donné à titre indicatif et susceptible d'être modifié.
- Dans le code C++ demandé vous vous efforcerez d'écrire du code correct et compilable, mais les petites typos ne seront pas sanctionnées.
- De même les problèmes d'include et de namespace ne sont pas pertinents (pas la peine de qualifier les `std::`, ni de citer tous les `#include`)
- Sauf indication contraire dans la question, on suppose également que **tous les appels système se passent bien**, on ne demande pas de contrôler les valeurs de retour (donc pas de `perror` ou `errno.h`).
- On vous fournit une annexe recensant la signature de certains appels système.

Le sujet est composé de sous sections indépendantes qu'on pourra traiter dans l'ordre qu'on souhaite.

I. Notions de File descriptor (6 points)

Question 1. (2,5 points) Implémentez la fonction `int copy(int fddest, int fdsrc)`. Cette fonction doit utiliser les appels système `read` et `write` pour transférer le contenu de `fdsrc` vers `fddest`. La fonction doit retourner le nombre total d'octets copiés, ou une valeur négative en cas d'erreur. Assurez la gestion des erreurs pour `read` et `write`, en affichant des messages appropriés avec `perror` et en retournant une valeur négative en cas d'échec. Les opérations de lecture et d'écriture doivent traiter les données par blocs de 1024 octets, sauf pour le dernier bloc.

On rappelle que `read` comme `write` rendent en général le nombre d'octets traités, zéro pour indiquer une fin de fichier, et une valeur négative traduisant une erreur décrite plus précisément à l'aide de `errno`.

fd

```
1 // Copy function
2 int copy(int fddest, int fdsrc) {
3     const int BUFFER_SIZE = 1024;
4     char buffer[BUFFER_SIZE];
5     ssize_t bytes_read, bytes_written;
6     ssize_t total_bytes = 0;
7 }
```

```

8  while ((bytes_read = read(fdsrc, buffer, BUFFER_SIZE)) > 0) {
9      bytes_written = write(fddest, buffer, bytes_read);
10     if (bytes_written != bytes_read) {
11         perror("write error");
12         return -1;
13     }
14     total_bytes += bytes_written;
15 }
16
17 if (bytes_read < 0) {
18     perror("read error");
19     return -1;
20 }
21
22 return total_bytes;
23 }

```

Barème (sur 100 %, à saisir tel quel):

- 10% declare a stack alloc bu er
- 25% use of read with the return value
- 10% catching any read error
- 25% write
- 10% catching write errors
- 10% correct loop to EOF/read returns 0
- 10% return total read

Question 2. (1,5 points) Ecrire un programme principal qui simule une version simplifiée de la commande “cat”. Si le programme est invoqué sans arguments, il lit sur son entrée standard et copie sur sa sortie standard (STDIN_FILENO, STDOUT_FILENO). Si des arguments sont fournis, il copie sur la sortie standard chacun des fichiers argument dans l’ordre.

Vous utiliserez la primitive “copy” de la question précédente, qu’on peut supposer résolue. Le programme doit se terminer et rendre *EXIT_FAILURE* si une copie échoue, sinon il rend *EXIT_SUCCESS*.

```

                                     fd
1  // Main program
2  int main(int argc, char *argv[]) {
3      if (argc == 1) {
4          // No arguments, copy from STDIN to STDOUT
5          if (copy(STDOUT_FILENO, STDIN_FILENO) < 0) {
6              return 1;
7          }
8      } else {
9          for (int i = 1; i < argc; ++i) {
10             int fd = open(argv[i], O_RDONLY);
11
12             if (fd < 0) {
13                 perror("open error");
14                 continue;
15             }
16
17             if (copy(STDOUT_FILENO, fd) < 0) {
18                 close(fd);

```

```

19         return 1;
20     }
21
22     close(fd);
23 }
24 }
25
26 return 0;
27 }

```

Barème (sur 100 %, à saisir tel quel):

- 20% gestion si pas d'arguments
- 20% utilisation de open dans une boucle
- 20% invocation de copy avec le fd
- 10% gestion d'erreurs de copy et return EXIT FAIL
- 20% invocations à close (si copy fail et sinon)
- 10 % la boucle sur les arguments

Question 3. (2 points) Pour chacune des situations suivantes, décrivez comment obtenir les descripteurs de fichier (file descriptors) adaptés à l'utilisation avec votre fonction `copy` implémentée précédemment. Précisez les appels système nécessaires pour créer la ressource correspondante et obtenir un descripteur de fichier valide pour `copy`. Vous préciserez si le descripteur obtenu peut être passé comme source ou destination de `copy`. Expliquez étape par étape les actions à réaliser.

1. Avec un tube nommé `“./montube”`
2. Avec un tube anonyme
3. Avec une socket TCP, côté client
4. Avec une socket TCP, côté serveur.

1. En travaillant avec un tube nommé `“./montube”`. On invoque `mkfifo` pour le créer, puis `“open”` avec les flags `O_RDONLY` ou `O_WRONLY`. Le ou est exclusif donc surtout pas `O_RDWR`. Créez le tube nommé en utilisant `mkfifo("./montube", 0666)`. Ensuite, ouvrez le tube nommé avec `open("./montube", O_RDONLY)` pour la lecture ou `open("./montube", O_WRONLY)` pour l'écriture. Ces invocations à `open` rendent des fd adaptés pour la source et la destination respectivement.
2. En travaillant avec un tube anonyme, on invoque `“pipe(int fds[2])”` puis on utilise `fd[0]` pour read et `fd[1]` pour write. Ces descripteurs peuvent être passés directement à la fonction `copy`.
3. En travaillant avec une socket TCP, côté client. On invoque `“socket(AF_INET,SOCK_STREAM,0)”` puis `“connect”`, ensuite on peut invoquer `“copy”` (en source ou destination).
4. En travaillant avec une socket TCP, côté serveur. Commencez par créer une socket avec `socket(AF_INET, SOCK_STREAM, 0)`, liez-la à une adresse avec `bind`, puis écoutez les connexions entrantes avec `listen`. Utilisez `accept` pour accepter une connexion entrante. Le descripteur de fichier retourné par `accept` peut être utilisé dans la fonction `copy` (comme source OU destination). On invoque `socket`, `bind`, `listen` et enfin `“accept”` pour obtenir la socket de communication qu'on pourrait passer à `copy` (comme source OU destination)

Barème.

1. Création avec `mkfifo` : 10%. Ouverture avec `open` et choix correct des flags : 10%. Explication claire de l'utilisation avec `copy` : 5%.
2. Utilisation correcte de `pipe` : 10%. Explication de l'utilisation des descripteurs : 10%, on précise lequel est en read ou write : 5 %

3. Création de la socket : 10%. Connexion avec connect : 10%. Explication de l'utilisation avec copy : 5%
4. Création de la socket serveur, bind et listen : 15 % (donc 5% chacun). Acceptation de la connexion avec accept : 10%.

II. Signaux (3 points)

Question 4. (3 points) Écrivez un programme C qui implémente le comportement suivant avec deux processus, un père et un fils :

- Le fils doit envoyer un signal SIGINT à son père puis se terminer
- Le père doit masquer le signal SIGINT, attendre la terminaison du fils, puis attendre de recevoir le signal SIGINT à l'aide de `sigsuspend` (ce qui va le terminer).

signals

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <signal.h>
5 #include <sys/types.h>
6 #include <sys/wait.h>
7
8 int main() {
9     sigset_t maskint;
10    // Bloquer SIGINT
11    sigemptyset(&maskint);
12    sigaddset(&maskint, SIGINT);
13    sigprocmask(SIG_BLOCK, &maskint, NULL);
14
15    // Créer un processus enfant
16    pid_t pid = fork();
17    if (pid == 0) {
18        // Processus enfant
19        // Envoyer SIGINT au processus parent
20        kill(getppid(), SIGINT);
21        printf("Le fils se termine.\n");
22        exit(0);
23    } else if (pid > 0) {
24        // Processus parent
25        // Attendre la fin du processus enfant
26        wait(NULL);
27
28        printf("Le père sait que le fils s'est terminé. Il se prépare à recevoir SIGINT\n");
29
30        sigset_t suspend_set;
31        // Préparer la suspension du processus
32        sigfillset(&suspend_set);
33        sigdelset(&suspend_set, SIGINT); // Ne pas bloquer SIGINT
34
35        // Suspendre le processus jusqu'à la réception d'un SIGINT
36        sigsuspend(&suspend_set);
37
38        printf("Processus parent terminé.\n");
39    } else {

```

```

40     perror("Échec de fork");
41     exit(1);
42 }
43
44 return 0;
45 }

```

Barème :

30% : 15 % on sigprocmask au début du programme, avant le fork. 15 % construction correcte du masque avec SIGINT à 1.

10 % le père fait le fork

20 % le fils kill son père avec getppid

10 % le père wait son fils, avant de sigsuspend

30 % : 15 % invocation à sigsuspend; 15 % construction du masque en visant SIGINT (i.e. il n'est PAS masqué).

III. Questions sur Sockets (2,5 points)

Question 5. (2,5 points) Répondez concisément et précisément à chacune des questions suivantes.

1. Dans quel(s) contexte(s) et dans quel but peut on croiser la ligne `bind(sockfd, (struct sockaddr *)&addr, sizeof(addr))` ?
2. Expliquez pourquoi dans cette invocations on a un cast “(struct sockaddr *)”. De quel(s) type(s) pourrait être “addr” en pratique ?
3. Comment fermer proprement une socket TCP après avoir fini de communiquer ?
4. Quels sont les étapes (appels à l’api) nécessaires pour émettre un paquet UDP vers une adresse de multicast ?
5. A l’aide de quel(s) invocations résoudre le nom du serveur “www.lip6.fr” vers une adresse IP utilisable dans l’api des sockets ? Sur quel système distribué sous jacent s’appuie cette invocation ?

1. Dans un serveur, qu’il soit TCP ou UDP ou autre (10 %), cette ligne demande au système de déclarer cette adresse comme accessible aux demandes de connexion/données (10%).
2. Car l’API `sockaddr` est générique et supporte plusieurs familles d’adresses (10%) comme `AF_UNIX`, `AF_INET`, `AF_INET6`... (10%)
3. Penser à faire un shutdown (10%) avant de faire le close (10%) est poli , pour permettre la fin de connection du protocole TCP.
4. Que ce soit une adresse de multicast ne change rien vu que c’est un envoi et pas une réception. Donc on veut invoquer `socket` (10%) puis `sendto` (10%).
5. On va utiliser “`getaddrinfo`” en passant ou pas des hints, et en itérant la liste rendue (10%). On donne quand même les points si on cite “`gethostbyname`” (10%) qui est moins flexible et obsolète mais qui marche ipv4/tcp. 10% pour la réponse sur le système distribué sous jacent qui est le DNS.

Barème (sur 100), 20 % par question.

IV. Producteur Consommateur en Mémoire partagée (6 points)

Question 6. (2,5 points) Ecrire un programme “producteur” qui doit :

1. Créer et allouer dans “/myshm” un segment de mémoire partagée de 4096 octets.

2. Créer et allouer dans “/mysem” un sémaphore nommé initialement à zéro.
3. Attendre une saisie de l'utilisateur sur l'entrée standard. On utilisera simplement la primitive `read` sur `STDIN_FILENO`.
4. S'assurer de placer la saisie de l'utilisateur dans le segment de mémoire partagée
5. Déverrouiller le sémaphore pour indiquer que la donnée est prête
6. Se terminer proprement (mais sans détruire les IPC créés).

producer

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <fcntl.h>
4  #include <sys/mman.h>
5  #include <semaphore.h>
6  #include <unistd.h>
7  #include <string.h>
8
9  int main() {
10     const char *shm_name = "/myshm";
11     const char *sem_name = "/mysem";
12     const int SIZE = 4096;
13     int shm_fd;
14     void *ptr;
15     sem_t *sem;
16
17     // Create the shared memory segment
18     shm_fd = shm_open(shm_name, O_CREAT | O_RDWR, 0666);
19     ftruncate(shm_fd, SIZE);
20     ptr = mmap(0, SIZE, PROT_WRITE, MAP_SHARED, shm_fd, 0);
21
22     // Create a new semaphore
23     sem = sem_open(sem_name, O_CREAT, 0666, 0);
24
25     // Write to the shared memory reading from STDIN
26     read(STDIN_FILENO, ptr, SIZE);
27
28     // Signal the semaphore to indicate data is ready
29     sem_post(sem);
30
31     printf("Data written to shared memory: %s\n", message);
32
33     // Cleanup
34     munmap(ptr, SIZE);
35     close(shm_fd);
36     sem_close(sem);
37
38     return 0;
39 }

```

Programme Producteur

Création et allocation du segment de mémoire partagée (/myshm) : Correctement créer et allouer le segment : 20% Configuration correcte de la taille (4096 octets) : 10%

Création et allocation du sémaphore nommé (/mysem) : Création correcte du sémaphore : 10% Initialisation initiale à zéro : 10%

Gestion de la saisie de l'utilisateur : Lecture correcte 20%

Déverrouillage du sémaphore et nettoyage : Déverrouillage correct du sémaphore : 10% Nettoyage approprié sans effacer les IPC : 20%

Question 7. (2 points) Ecrire un programme “consommateur” qui doit :

1. Accéder aux ressources “/myshm” et “/mysem” créés par l’autre processus
2. Attendre que le producteur ait indiqué que la donnée est disponible dans la mémoire partagée
3. Recopier la saisie de l’utilisateur sur la sortie standard
4. Se terminer proprement en effaçant les IPC créés.

```

                                     consumer
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <fcntl.h>
4  #include <sys/mman.h>
5  #include <semaphore.h>
6  #include <unistd.h>
7
8  int main() {
9      const char *shm_name = "/myshm";
10     const char *sem_name = "/mysem";
11     const int SIZE = 4096;
12
13     // Open the existing shared memory segment
14     int shm_fd = shm_open(shm_name, O_RDONLY, 0666);
15     char * ptr = mmap(0, SIZE, PROT_READ, MAP_SHARED, shm_fd, 0);
16
17     // Open the existing semaphore
18     sem_t * sem = sem_open(sem_name, 0);
19
20     // Wait for the semaphore signal
21     sem_wait(sem);
22
23     // Read from the shared memory
24     printf("Data read from shared memory: %s\n", ptr);
25
26     // Cleanup
27     munmap(ptr, SIZE);
28     close(shm_fd);
29     sem_close(sem);
30
31     sem_unlink(sem_name);
32     shm_unlink(shm_name);
33
34     return 0;
35 }

```

Barème :

shmopen + mmap : 20 %
semopen : 10 %
semwait : 10%
lecture de la data : 10 %
munmap/close sur shm : 20%
semclose : 10%
semunlink, shmunlink : 20%

Question 8. (1,5 point) Sans le programmer entièrement, expliquez ce qu’il faudrait modifier dans ces deux programmes pour que le sémaphore soit hébergé dans le segment de mémoire partagée

désigné par “/myshm”.

40% On déclare un struct avec un champ data et un sem_t pour le sémaphore.
 40% On le sem_init à la construction
 20% On le sem_destroy avant de désallouer le segment.
 Sinon on s'en sert comme du sémaphore nommé précédent.

V. Producteur Consommateur en Multi-Thread (5,5 points)

On suppose le fichier suivant, à compléter.

mt.cpp

```

1  #include <iostream>
2  #include <thread>
3  #include <mutex>
4  #include <condition_variable>
5  #include <queue>
6
7  std::mutex mtx;
8  std::condition_variable cv;
9  std::queue<int> buffer;
10 const unsigned int maxBufferSize = 10;
11
12 // Producer function
13 void producer(int count) {
14 }
15
16 // Consumer function
17 void consumer() {
18 }
19
20 int main() {
21     return 0;
22 }
```

Question 9. (2,25 points) Compléter la fonction “void producer (int count)” qui insère un par un les entiers de 0 à *count* – 1 dans la queue, en s’assurant que la taille de la queue n’excède jamais “maxBufferSize”. On rappelle qu’on peut insérer un entier *i* dans la queue avec “buffer.push(i)” et que le remplissage de la queue est donné par “buffer.size()”.

Bu er

```

1  // Producer function
2  void producer(int count) {
3      for (int i = 0; i < count; ++i) {
4          {
5              std::unique_lock<std::mutex> lock(mtx);
6              cv.wait(lock, [] { return buffer.size() < maxBufferSize; });
7              std::cout << "Produced: " << i << std::endl;
8              buffer.push(i);
9          } // scope pour notify hors du lock
10         cv.notify_all();
11     }
12 }
```


Question 10. (2,25 points) Compléter la fonction “void consumer()” qui consomme un par un les éléments de la queue dans une boucle infinie, en attendant aux moments opportuns qu’il y en ait à lire. On rappelle qu’on peut retirer un entier i dans la queue avec “ $i = \text{buffer.front()}; \text{buffer.pop()};$ ” quand la queue est non vide.

Bu er

```

1 // Consumer function
2 void consumer() {
3     while (true) {
4         {
5             std::unique_lock<std::mutex> lock(mtx);
6             cv.wait(lock, [] { return !buffer.empty(); });
7             int value = buffer.front();
8             buffer.pop();
9             std::cout << "Consumed: " << value << std::endl;
10        } // scope pour notify hors du lock
11        cv.notify_all();
12    }
13 }

```

Déclaration de la boucle while : 10%

unique_lock verrou (mutex) : 20% + 10% si dans un scope dans le while

Attente conditionnelle : 30%

Consommation dans la queue : 10%

cv.notify_all() hors scope du unique_lock (ou après un unlock si pas de scope) : 20%, 10% sinon

Question 11. (1 points) Compléter le main, qui doit lancer un thread producteur avec *count* = 50 et un thread consommateur, puis détacher le thread consommateur mais attendre la fin du producteur avant de se terminer.

Bu er

```

1 int main() {
2     std::thread producerThread(producer, 50);
3     std::thread consumerThread(consumer);
4
5     producerThread.join();
6     consumerThread.detach(); // Detached to allow continuous consumption
7
8     return 0;
9 }

```

Barème Ajusté pour le main

Création des threads : Création correcte du thread producteur : 30% Création correcte du thread consommateur : 20%

Gestion des threads : Utilisation correcte de join pour le thread producteur : 20% Utilisation correcte de detach pour le thread consommateur : 30%

Annexe

Signature par ordre alphabétique de certains appels système Posix utilisés dans l'UE.

```

int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
unsigned int alarm(unsigned int seconds);
int bind(int socket, const struct sockaddr *address, socklen_t address_len);
int close(int fd);
int dup(int oldfd);
int dup2(int oldfd, int newfd);
int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
pid_t fork(void);
int fstat(int fd, struct stat *buf);
int ftruncate(int fd, off_t length);
void freeaddrinfo(struct addrinfo *res);
int getaddrinfo(const char *node, const char *service, const struct addrinfo *hints,
                struct addrinfo **res);
pid_t getpid(void);
pid_t getppid(void);
uint32_t htonl(uint32_t hostlong);
uint16_t htons(uint16_t hostshort);
int kill(pid_t pid, int sig);
int listen(int sockfd, int backlog);
void *memcpy(void *dest, const void *src, size_t n);
void *memset(void *s, int c, size_t n);
int mkfifo(const char *pathname, mode_t mode);
void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);
int munmap(void *addr, size_t length);
uint32_t ntohl(uint32_t netlong);
uint16_t ntohs(uint16_t netshort);
int open(const char *pathname, int flags);
int open(const char *pathname, int flags, mode_t mode);
int pipe(int pipefd[2]);
ssize_t read(int fd, void *buf, size_t count);
ssize_t recv(int sockfd, void *buf, size_t len, int flags);
ssize_t recvfrom(int sockfd, void *buf, size_t len, int flags,
                 struct sockaddr *src_addr, socklen_t *addrlen);
int select(int nfds, fd_set *readfds, fd_set *writefds, fd_set *exceptfds, struct timeval *timeout);
int sem_destroy(sem_t *sem);
int sem_init(sem_t *sem, int pshared, unsigned int value);
sem_t *sem_open(const char *name, int oflag);
sem_t *sem_open(const char *name, int oflag, mode_t mode, unsigned int value);
int sem_post(sem_t *sem);
int sem_wait(sem_t *sem);
ssize_t send(int sockfd, const void *buf, size_t len, int flags);
ssize_t sendto(int sockfd, const void *buf, size_t len, int flags,
               const struct sockaddr *dest_addr, socklen_t addrlen);
int shm_open(const char *name, int oflag, mode_t mode);
int shm_unlink(const char *name);
int shutdown(int socket, int how);
int sigsuspend(const sigset_t *mask);
int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact);
int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
int socket(int domain, int type, int protocol);
int stat(const char *path, struct stat *buf);
int unlink(const char *pathname);
pid_t wait(int *wstatus);
pid_t waitpid(pid_t pid, int *wstatus, int options);
ssize_t write(int fd, const void *buf, size_t count);

```